

1.操作与落地

1.1.数据库基础知识

1.1.1.范式化设计

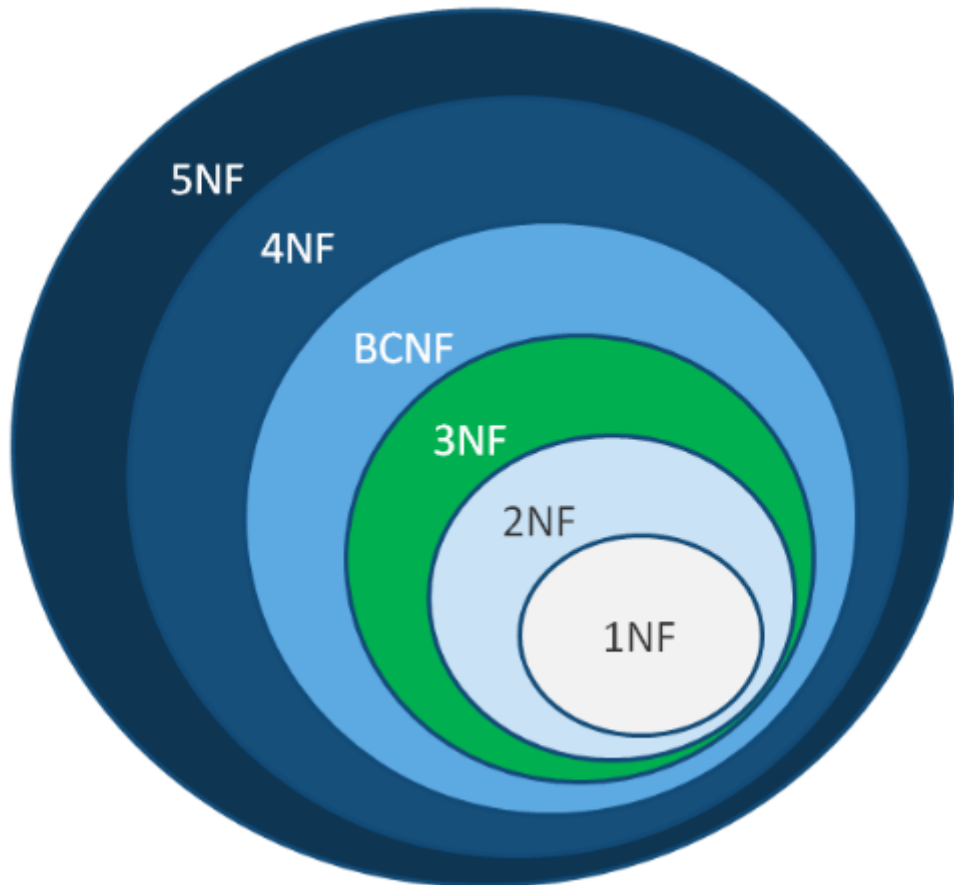
1.1.1.1.什么是范式

范式来自英文Normal Form，简称NF。

实际上你可以把它粗略地理解为 **一张数据表的表结构所符合的某种设计标准的级别**。就像家里装修买建材，最环保的是E0级，其次是E1级，还有E2级等等

目前关系数据库有六种范式：第一范式（1NF）、第二范式（2NF）、第三范式（3NF）、巴斯-科德范式（BCNF）、第四范式(4NF)和第五范式5NF，又称完美范式）。

满足最低要求的范式是第一范式（1NF），在第一范式的基础上进一步满足更多规范要求的称为第二范式（2NF），其余范式以次类推。一般来说，数据库只需满足第三范式(3NF)就行了。



1.1.1.2.第一范式（1NF）

定义：属于第一范式关系的所有属性都不可再分，即数据项不可分。

理解：第一范式强调数据表的原子性，是其他范式的基础。一张表有一个name-age列，这个列具有两个属性，一个name,一个 age，所以不符合第一范式，我们把它拆分成两列name和age，这张表就符合第一范式关系。

id	name-age		id	name	age
1	张三-18	➤	1	张三	18
2	李四-23		2	李四	23

你在关系型数据库管理系统（RDBMS），例如SQL Server，Oracle，MySQL中创建数据表的时候，1NF是所有关系型数据库设计的最基本要求。

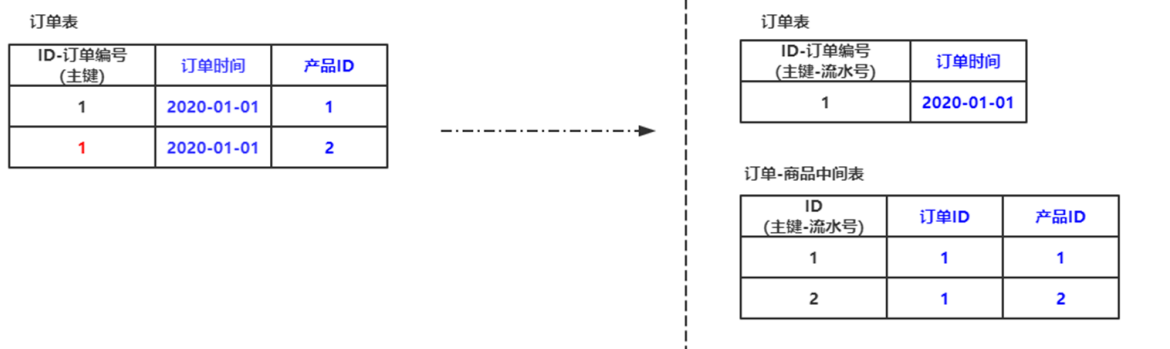
第一范式详细的要求如下：

- 1、每一列属性都是不可再分的属性值，确保每一列的原子性；
- 2、两列的属性相近或相似或一样，尽量合并属性一样的列，确保不产生冗余数据；
- 3、单一属性的列为基本数据类型构成；
- 4、设计出来的表都是简单的二维表。

1.1.1.2.第二范式（2NF）

第二范式（2NF）是在第一范式（1NF）的基础上建立起来的，即满足第二范式（2NF）必须先满足第一范式（1NF）。

第二范式（2NF）要求实体的属性完全依赖于主关键字。



名	类型	长度	小数点	不是 null	
id	int	11	0	☒	🔑 1
create_time	datetime	0	0	☐	
product_id	int	11	0	☒	🔑 2

id	create_time	product_id
1	2022-04-30 15:06:21	1
1	2022-04-30 15:06:33	2

以上这张表不符合第二范式（2NF），虽然有主键，但是实体的属性不完全依赖于主关键字。

所谓完全依赖是指不能存在仅依赖主关键字一部分的属性，如果存在，那么这个属性和主关键字的这一部分应该分离出来形成一个新的实体，新实体与原实体之间是一对多的关系。

id	create_time
1	2022-04-30 15:15:37

op_id	order_id	product_id
1	1	1
2	1	2

设计成两张表，主键分别是id和op_id，这样就符合第二范式（2NF）。

1.1.1.3.第三范式（3NF）

满足第三范式（3NF）必须先满足第二范式（2NF）；

第三范式（3NF）要求一个数据库表中不包含已在其它表中包含的非主关键字信息，即数据不能存在传递关系，即每个属性都跟主键有直接关系而不是间接关系。

订单表

ID-订单编号 (主键)	订单时间	产品ID	产品名称
1	2020-01-01	1	机械键盘
2	2020-01-02	2	蓝牙音响

产品表

产品ID	产品名称
1	机械键盘
2	蓝牙音响
3	蓝牙鼠标

这里如果产品ID或产品名称变化会发生什么情况？所以以上不符合第三范式（3NF）

订单表

ID-订单编号 (主键)	订单时间	产品ID
1	2020-01-01	1
2	2020-01-02	2

以上订单表就符合第三范式

1.1.2.反范式化设计

完全符合范式化的设计真的完美无缺吗？很明显在实际的业务查询中会大量存在着表的关联查询，而表设计都做成了范式化设计（甚至很高的范式），大量的表关联很多的时候非常影响查询的性能。

反范式化就是违反范式化设计：

- 1、为了性能和读取效率而适当的违反对数据库设计范式的要求；
- 2、为了查询的性能，允许存在部分（少量）冗余数据

换句话说反范式化就是使用空间来换取时间。

1.1.3.范式化和反范式对比

	范式设计	反范式设计
更新操作	快	慢
数据重复度	低	高
内存占用	小	大
查询表关联	多	少
查询索引命中	较少命中	更多命中

- 1、范式化的更新操作通常比反范式化要快（字段较少）。
- 2、当数据较好地范式化时，就只有很少或者没有重复数据，所以只需要修改更少的数据。
- 3、范式化的表通常更小，所以占据的内存更少。
- 4、范式化设计的缺点是通常需要关联，稍微复杂一些的查询语句在符合范式的表上可能都需要至少一次关联，也许更多。
- 5、复杂一些的查询语句也可能使一些索引策略无效。例如，范式化可能将列存放在不同的表中，而这些列如果在一个表中本可以属于同一个索引。

1.1.4.项目中常见的反范式实现

范式化和反范式化的各有优劣，**怎么选择最佳的设计？小孩子才做选择，我们全都要！！**

1.1.4.1.缓存与汇总数据

“缓存”来表示存储那些可以比较简单地从其他表获取数据的表。

比如从父表冗余一些数据到子表的。前面我们看到的分类信息放到商品表里面进行冗余存放就是典型的例子。

“汇总”则保存的是使用GROUP BY语句聚合数据的表。

如果需要显示每个用户发了多少消息，可以每次执行一个对用户发送消息进行count的子查询来计算并显示它，也可以在user表用户中建一个消息发送数目的专门列，每当用户发新消息时更新这个值。

在使用缓存表和汇总表时，有个关键点是如何维护缓存表和汇总表中的数据，常用的有两种方式，实时维护数据和定期重建，这个取决于应用程序，不过一般来说，缓存表用实时维护数据更多点，往往在一个事务中同时更新数据本表和缓存表，汇总表则用定期重建更多，使用定时任务对汇总表进行更新。

1.1.4.2.计数器表设计

计数器表在Web应用中很常见。比如网站点击数、用户的朋友数、文件下载次数等。对于高并发下的处理，首先可以创建一张独立的表存储计数器，这样可使计数器表小且快，并且可以使用一些更高级的技巧。

id	count_name	count
1	check	80

名	类型	长度	小数点	不是 null	
id	int	11	0	☒	 1
count_name	varchar	255	0	☐	
count	int	11	0	☐	

```
update COUNT SET count=count+1 WHERE id=1;
```

比如假设有一个计数器表，只有一行数据，记录网站的点击次数，网站的每次点击都会导致对计数器进行更新，问题在于，对于任何想要更新这一行的事务来说，这条记录上都有一个全局的互斥锁(mutex)。这会使得这些事务只能串行执行，会严重限制系统的并发能力。

怎么改进呢？可以将计数器保存在多行中，每次随机选择一行进行更新。在具体实现上，可以增加一个槽 (slot) 字段，然后预先在这张表增加100行或者更多数据，当对计数器更新时，选择一个随机的槽 (slot) 进行更新即可。

id	count_name	slot	count
1	check	1	10
1	check	2	14
1	check	3	24

名	类型	长度	小数点	不是 null	
id	int	11	0	☒	 1
count_name	varchar	255	0	☐	
slot	int	11	0	☒	 2
count	int	11	0	☐	

```
update count_s SET count=count+1 WHERE id=1 AND slot =1;
update count_s SET count=count+1 WHERE id=1 AND slot =2;
update count_s SET count=count+1 WHERE id=1 AND slot =3;
```

1.1.4.字段数据类型优化

MySQL支持的数据类型非常多，选择正确的数据类型对于获得高性能至关重要。不管存储哪种类型的数据，下面几个简单的原则都有助于做出更好的选择。

1.1.4.1.字段优化基本原则

- 更小的通常更好

一般情况下,应该尽量使用可以正确存储数据的最小数据类型。更小的数据类型通常更快，因为它们占用更少的磁盘、内存和CPU缓存，并且处理时需要的CPU周期也更少。

比如：是有一个类型既可以用字符串也可以使用整型，**优先选择整型**。因为字符串牵涉到了字符集及校对规则等。

- 简单就好

简单数据类型的操作通常需要更少的CPU周期。例如，整型比字符操作代价更低，因为字符集和校对规则(排序规则)使字符比较比整型比较更复杂。比如应该使用MySQL内建的类型而不是字符串来存储日期和时间。

- 尽量避免NULL

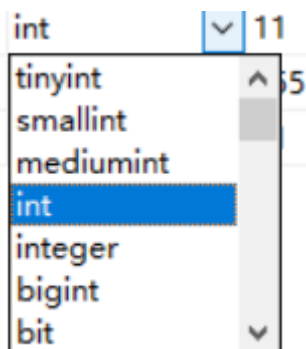
通常情况下最好指定列为NOT NULL，除非真的需要存储NULL值。

如果查询中包含可为NULL的列，对MySQL来说更难优化，因为可为NULL的列使得索引、索引统计和值比较都更复杂。可为NULL的列会使用更多的存储空间，在MySQL里也需要特殊处理。当可为NULL的列被索引时，每个索引记录需要一个额外的字节。

通常把可为NULL的列改为NOT NULL带来的性能提升比较小，所以（调优时）没有必要首先在现有schema中查找并修改掉这种情况，除非确定这会导致问题。但是，如果计划在列上建索引，就应该尽量避免设计成可为NULL的列。

1.1.4.2.Int/整数类型

存储整数，可以使用这几种整数类型:TINYINT，SMALLINT，MEDIUMINT，INT，BIGINT。分别使用8，16，24，32，64位存储空间，也就是1、2、3、4、8个字节。它们可以存储的值的范围请自行计算。



同时整数类型有可选的 UNSIGNED属性，表示不允许负值，这大致可以使正数的上限提高一倍。例如 TINYINT UNSIGNED可以存储的范围是0255，而TINYINT的存储范围是-128127。

有符号和无符号类型使用相同的存储空间，并具有相同的性能，因此可以根据实际情况选择合适的类型。

另外integer和int存储及大小没有任何差别，只是为了业务上区分。

> count	int	11	0	☐	
---------	-----	----	---	--------------------------	--

默认:

NULL

注释:

☐ 自动递增

☐ 无符号

☐ 填充零

MySQL可以为整数类型指定宽度，例如INT(11)，对大多数应用这是没有意义的，它不会限制值的合法范围,只是规定了MySQL的一些交互工具（例如MySQL命令行客户端)用来显示字符的个数。对于存储和计算来说，INT(1)和INT(20)是相同的。

名	类型	长度	小数点	不是 null	
id	int	11	0	☒	1
...	...	...	...	☐	

MySQL中没有long型，对应的只有bigint

1.1.4.3.实数类型

实数是带有小数部分的数字。MySQL既支持精确类型的存储DECIMAL类型，也支持不精确类型存储FLOAT和 DOUBLE类型（浮点类型）。DECIMAL类型用于存储精确的小数，本质上MySQL是以字符串形式存放的。所以CPU不支持对DECIMAL的直接计算，只是在MySQL中自身实现了DECIMAL的高精度计算。相对而言，CPU直接支持原生浮点计算，所以浮点运算明显更快。

浮点类型在存储同样范围的值时，通常比DECIMAL使用更少的空间。FLOAT使用4个字节存储，DOUBLE占用8个字节，DECIMAL里面存储65个数字，DECIMAL对于列的空间消耗比较大，另外DOUBLE比 FLOAT有更高的精度和更大的范围。

如何选择?

在精度不敏感和需要快速运算的时候，选择FLOAT和 DOUBLE。

应该尽量只在对小数进行精确计算时才使用DECIMAL，例如存储财务或金融数据。

但在数据量比较大的而且要求精度时，可以考虑使用BIGINT代替DECIMAL，将需要存储的货币单位根据小数的位数乘以相应的倍数即可。假设要存储财务数据精确到万分之一分，则可以把所有金额乘以一百万,然后将结果存储在BIGINT里，这样可以同时避免浮点存储计算不精确和 DECIMAL精确计算代价高的问题。

1.1.4.4.字符串类型

MySQL支持多种字符串类型，包括VARCHAR和CHAR类型、BLOB和TEXT类型、ENUM（枚举）和SET类型。

VARCHAR和 CHAR是两种最主要的字符串类型。

VARCHAR

VARCHAR类型用于存储可变长字符串，是最常见的字符串数据类型。它比定长类型更节省空间，因为它仅使用必要的空间（例如，越短的字符串使用越少的空间）。在内部实现上，既然是变长，VARCHAR需要使用1或2个额外字节记录字符串的长度，如果列的最大长度小于或等于255字节，则只使用1个字节表示，否则使用2个字节。

VARCHAR节省了存储空间，所以对性能也有帮助。但是，由于行是变长的，在UPDATE时新值比旧值长时，使行变得比原来更长，这就可能导致需要做额外的工作。如果一个行占用的空间增长，并且在页内没有更多的空间可以存储，在这种情况下，MyISAM会将行拆成不同的片段存储，InnoDB则需要分裂页来使行可以放进页内。

CHAR

CHAR类型是定长的，MySQL总是根据定义的字符串长度分配足够的空间。当存储CHAR值时，MySQL会删除所有的末尾空格，CHAR值会根据需要采用空格进行填充以方便比较。

CHAR与VARCHAR如何选择？

在CHAR和VARCHAR的选择上，这些情况下使用VARCHAR是合适的：

字符串列的最大长度比平均长度大很多，列的更新很少；使用了像UTF-8这样复杂的字符集，每个字符都使用不同的字节数进行存储。

CHAR适合存储很短的字符串，或者所有值定长或都接近同一个长度。例如，CHAR非常适合存储密码的MD5值，因为这是一个定长的值。对于经常变更的数据，CHAR也比VARCHAR更好，因为定长的CHAR类型不容易产生碎片。

对于非常短的列，CHAR比VARCHAR在存储空间上也更有效率。例如用CHAR(1)来存储只有Y和N的值，如果采用单字节字符集只需要一个字节，但是VARCHAR(1)却需要两个字节，因为还有一个记录长度的额外字节。

另外，使用VARCHAR(5)和VARCHAR(200)存储'hello'在磁盘空间上开销是一样的。我们随便选择一个就好？应该使用更短的列，为什么？

事实证明有很大的优势。更长的列会消耗更多的内存，因为MySQL通常会分配固定大小的内存块来保存内部值。尤其是使用内存临时表进行排序或操作时会特别糟糕。在利用磁盘临时表进行排序时也同样糟糕。

所以最好的策略是只分配真正需要的空间。

1.1.4.5.BLOB和TEXT类型

BLOB和TEXT都是为存储很大的数据而设计的字符串数据类型，分别采用二进制和字符方式存储。

与其他类型不同，MySQL把每个BLOB和TEXT值当作一个独立的对象处理。存储引擎在存储时通常会做特殊处理。当BLOB和TEXT值太大时，InnoDB会使用专门的“外部”存储区域来进行存储，此时每个值在行内需要1~4个字节存储一个指针，然后在外部存储区域存储实际的值。

BLOB和TEXT家族之间仅有的不同是BLOB类型存储的是二进制数据，没有排序规则或字符集，而TEXT类型有字符集和排序规则。

使用BLOB和TEXT要慎重：

- (1) BLOB和TEXT值会引起一些性能问题，所以尽量避免使用BLOB和TEXT类型；
- (2) 一定要用，建议把BLOB或TEXT列分离到单独的表中；

(3) 在不必要的时候避免检索大型的 BLOB或TEXT值。例如，SELECT *查询就不是很好的想法，除非能够确定作为约束条件的WHERE子句只会找到所需要的数据行。否则，很可能毫无目的地在网络上传输大量的值。建议可以搜索索引列，决定需要的哪些数据行，然后从符合条件的数据行中检索BLOB或 TEXT 值；

(4) 还可以使用合成的(Synthetic)索引来提高大文本字段(BLOB或TEXT)的查询性能。简单来说，合成索引就是根据大文本字段的内容建立一个散列值，并把这个值存储在单独的数据列中，接下来就可以通过检索散列值找到数据行了。但是，要注意这种技术只能用于精确匹配的查询（散列值对于类似“<”或“>=”等范围搜索操作符是没有用处的）。可以使用MD5函数生成散列值，也可以使用SHA1(或CRC32)，或者使用自己的应用程序逻辑来计算散列值。

1.1.4.6.枚举类型

如果表中的字段的取值是固定几个字符串，可以使用枚举列代替常用的字符串类型。

枚举列可以把一些不重复的字符串存储成一个预定义的集合。MySQL在存储枚举时非常紧凑，会根据列表值的数量压缩到一个或者两个字节中，MySQL在内部会将每个值在列表中的位置保存为整数，这样的话可以让表的大小大为缩小。

具体枚举使用见官网地址：<https://dev.mysql.com/doc/refman/5.7/en/enum.html>

```
CREATE TABLE
enum_test(e ENUM(' fish', 'apple', 'dog') NOT NULL);

INSERT INTO
enum_test(e) VALUES(1),(2),(3);
```

但是要注意，

- 1) 因为枚举列实际存储为整数，而不是字符串，所以不要使用数字作为ENUM枚举常量，这种双重性很容易导致混乱，例如ENUM('1','2','3')。
- 2) 枚举字段是按照内部存储的整数而不是定义的字符串进行排序的，所以尽量按照需要的顺序来定义枚举列。

1.1.4.7.日期和时间类型

MySQL可以使用许多类型来保存日期和时间值，例如YEAR和 DATE以及DATETIME和TIMESTAMP。MySQL能存储的最小时间粒度为秒。

datetime 存储日期范围：1001年~9999年

timestamp 存储日期范围：1970年~2038年,并且跟时区有关系。

如果需要存储比秒更小粒度的日期和时间值怎么办？MySQL目前没有提供合适的数据类型，但是可以使用自己的存储格式：可以使用BIGINT类型存储微秒级别的时间戳，或者使用DOUBLE存储秒之后的小数部分。

1.1.5.命名规范

1、可读性原则

数据库、表、字段的命名要遵守可读性原则，尽可能少使用或者不使用缩写。

对象的名字应该能够描述它所表示的对象。例如：表的名称应该能够体现表中存储的数据内容，最好是遵循“业务名称_表的作用”；对于存储过程存储过程应该能够体现存储过程的功能。库名与应用名称尽量一致。

表达是与否概念的字段，应该使用is_xxx的方式命名，数据类型是unsigned tinyint（1表示是，0表示否）。

2、表名、字段名必须使用小写字母或数字，禁止出现数字开头，禁止两个下划线中间只出现数字。数据库字段名的修改代价很大，因为无法进行预发布，所以字段名称需要慎重考虑。

说明：MySQL在Windows下不区分大小写，但在Linux下默认是区分大小写。因此，数据库名、表名、字段名，都不允许出现任何大写字母，避免节外生枝。

3、表名不使用复数名词

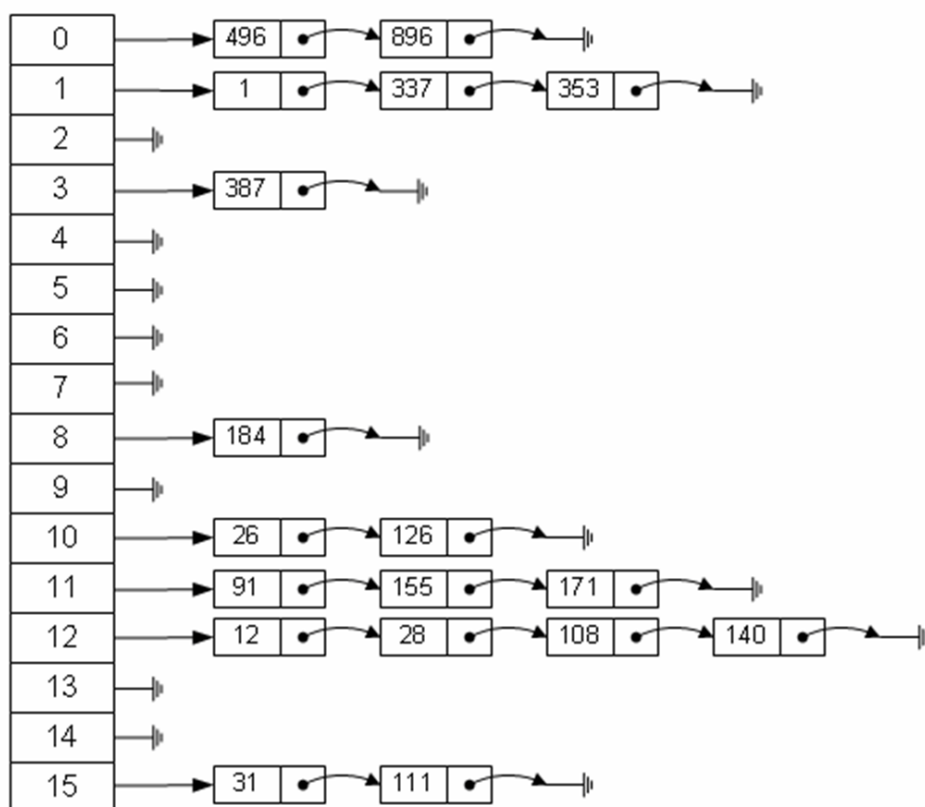
4、数据库、表、字段的命名禁用保留字，如desc、range、match之类

6、主键索引名为pk_字段名；唯一索引名为uk_字段名；普通索引名则为idx_字段名。

在面试过程中涉及到设计表的时，如果命名不规范，必定是一个很大的扣分项。

1.1.6MySQL中的索引

MySQL官方对索引的定义为：索引（Index）是帮助MySQL高效获取数据的数据结构。可以得到索引的本质：**索引是数据结构**。InnoDB存储引擎支持以下几种常见的索引：B+树索引、全文索引、哈希索引，其中比较关键的是B+树索引



那为什么HashMap不适合做数据库索引？

- 1、hash表只能匹配是否相等，不能实现范围查找；
- 2、当需要按照索引进行order by时，hash值没办法支持排序；
- 3、组合索引可以支持部分索引查询，如(a,b,c)的组合索引，查询中只用到了a和b也可以查询的，如果使用hash表，组合索引会将几个字段合并hash，没办法支持部分索引；
- 4、当数据量很大时，hash冲突的概率也会非常大。

1.1.7.B+Tree

B+树索引就是传统意义上的索引，这是目前关系型数据库系统中查找最常用和最为有效的索引。B+树索引的构造类似于二叉树，根据键值（Key Value）快速找到数据。注意B+树中的B不是代表二叉(binary)，而是代表平衡(balance)，因为B+树是从最早的平衡二叉树演化而来，但是B+树不是一个二叉树。

在讲二叉树之前，我们必须了解一下**二分查找**：

二分查找法（binary search）也称为折半查找法，用来查找一组有序的记录数组中的某一记录。

在以下数组中找到数字48对应的下标

数值	5	10	19	21	31	37	42	48	50	52
下标	0	1	2	3	4	5	6	7	8	9

通过3次二分查找 就找到了我们所要的数字，而顺序查找需8次。

对于上面10个数来说，顺序查找平均查找次数为 $(1+2+3+4+5+6+7+8+9+10)/10=5.5$ 次。而二分查找法为 $(4+3+2+4+3+1+4+3+2+3)/10=2.9$ 次。在最坏的情况下，顺序查找的次数为10，而二分查找的次数为4。

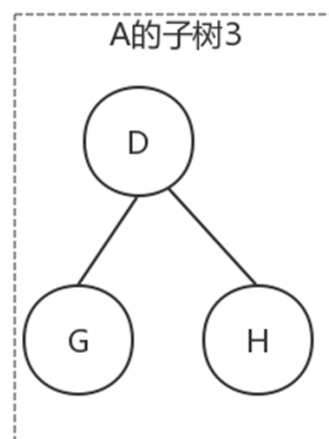
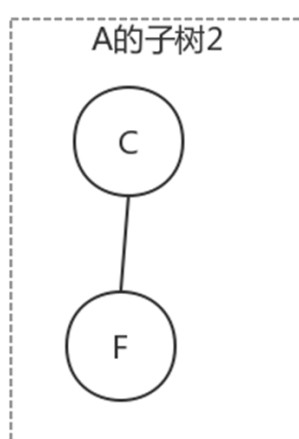
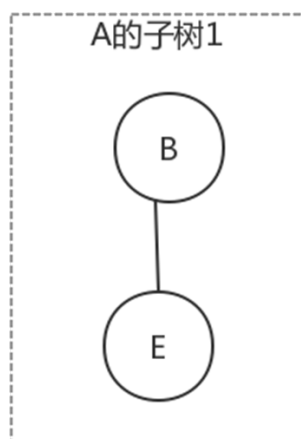
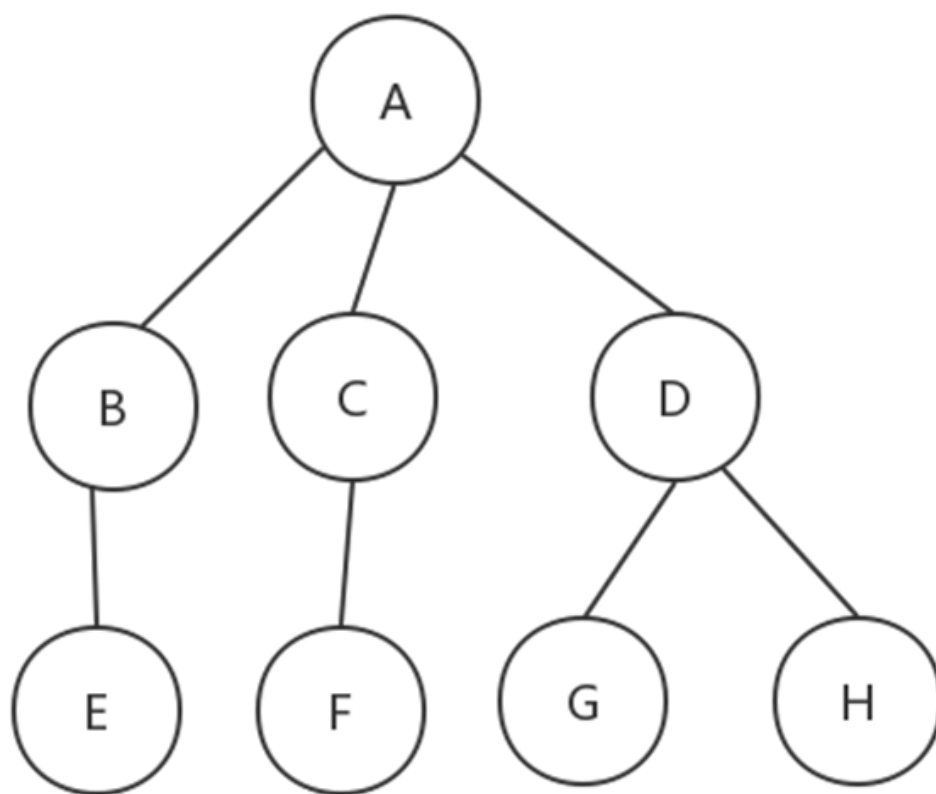
所以为了索引查找的高效性，我们引入了二叉查找树。

1.1.7.1.二叉树

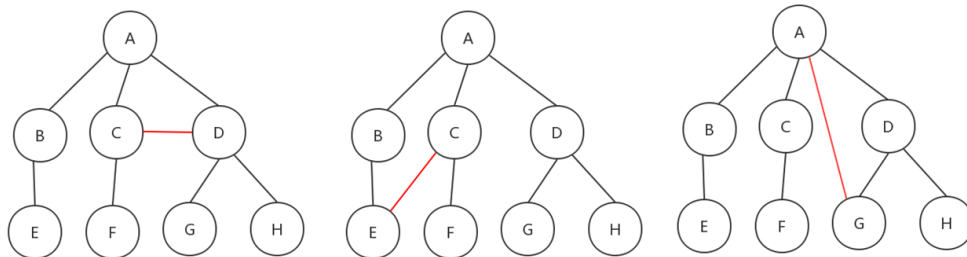
1.1.7.1.1.树(Tree)

N个结点构成的有限集合。

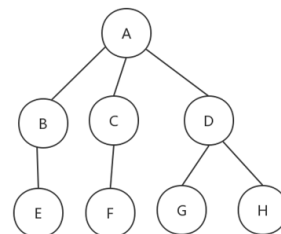
- 树中有一个称为“根(Root)”的特殊结点
- 其余结点可分为M个互不相交的树，称为原来结点的“子树”



1.1.7.1.2.树与非树

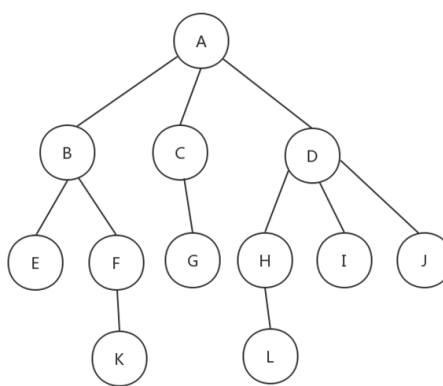


- 子树是不相交的
- 除了根结点之外，每个结点之有且只有一个父结点
- 一个N个结点的树只有N-1条边



1.1.7.1.3.树的一些基本术语

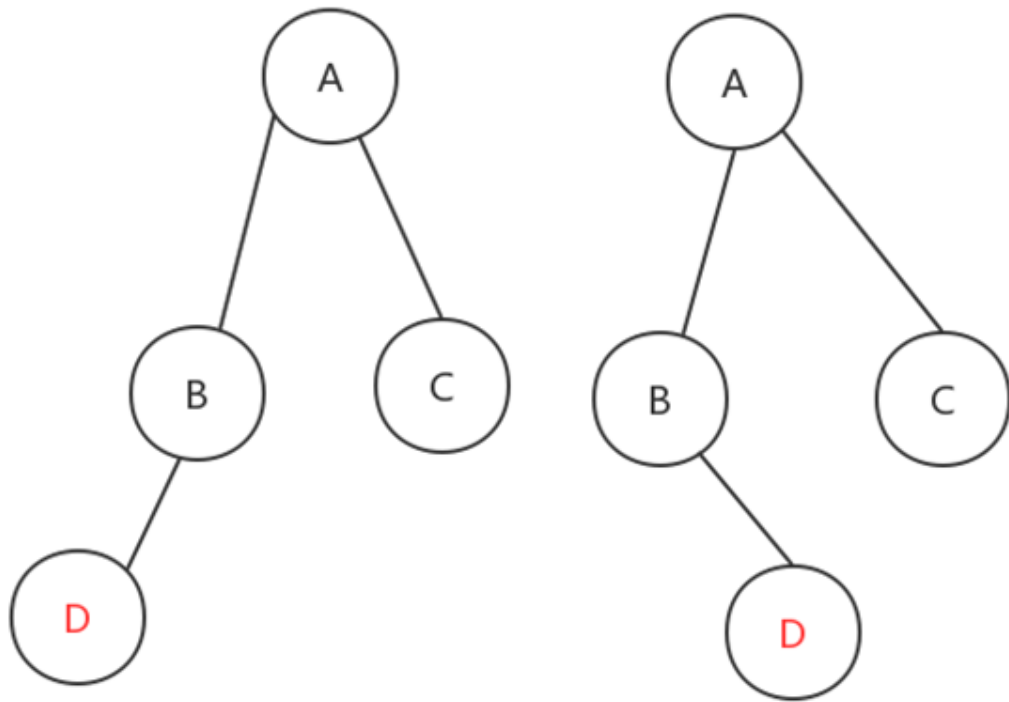
- 结点的度：结点的子树个数
- 树的度：树中所有结点中最大的度
- 叶结点：度为0的结点
- 父结点：有子树的结点是其子树的根结点的父结点
- 子结点：若A是B的父结点，B就是A的子结点
- 树的深度：树中所有结点中最大的层次是这棵树的深度



1.1.7.1.4.二叉树

度为2的树（也可称之为阶）：（树的度：树中所有结点中最大的度。结点的度：结点的子树个数）

子树有左右顺序之分：

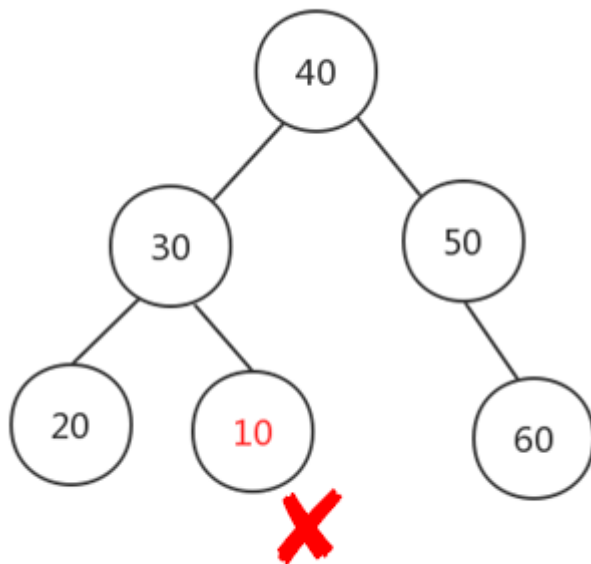
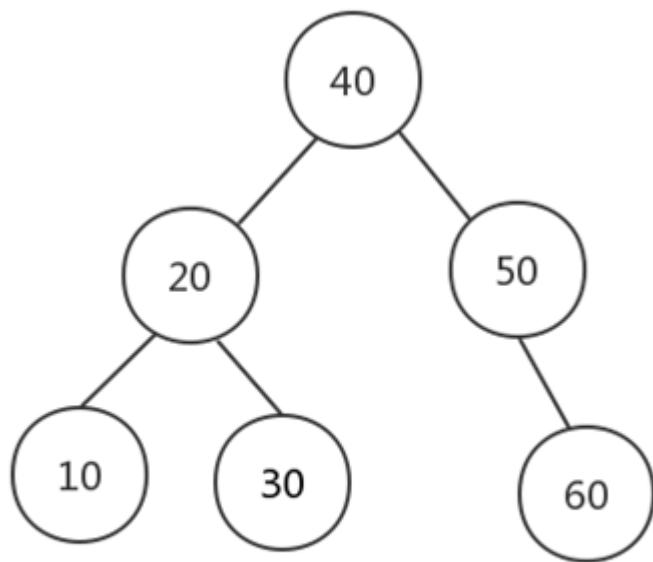


这是两个不同的二叉树（左右之分）

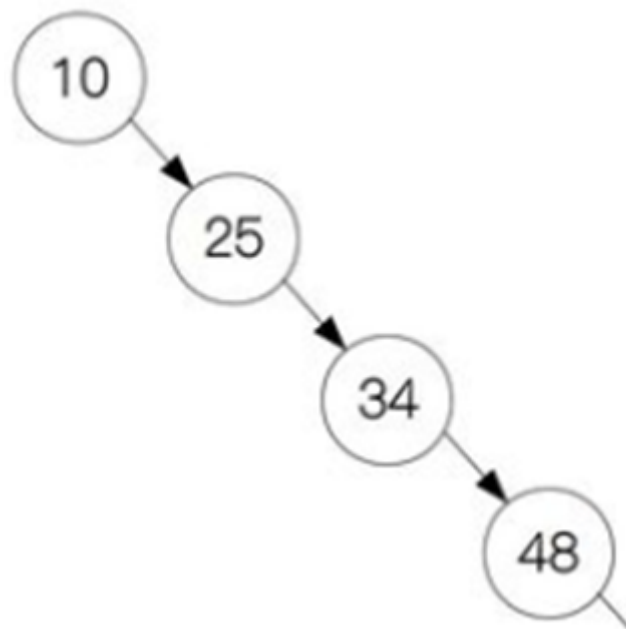
1.1.7.1.5.二叉查找(搜索)树

二叉查找树首先肯定是个二叉树，除此之外还符合以下几点：

- 左子树的所有的值小于根节点的值
- 右子树的所有的值大于或等于根节点的值
- 左、右子树满足以上两点



但是二叉查找树，如果设计不良，完全可以变成一颗极不平衡的二叉查找树：

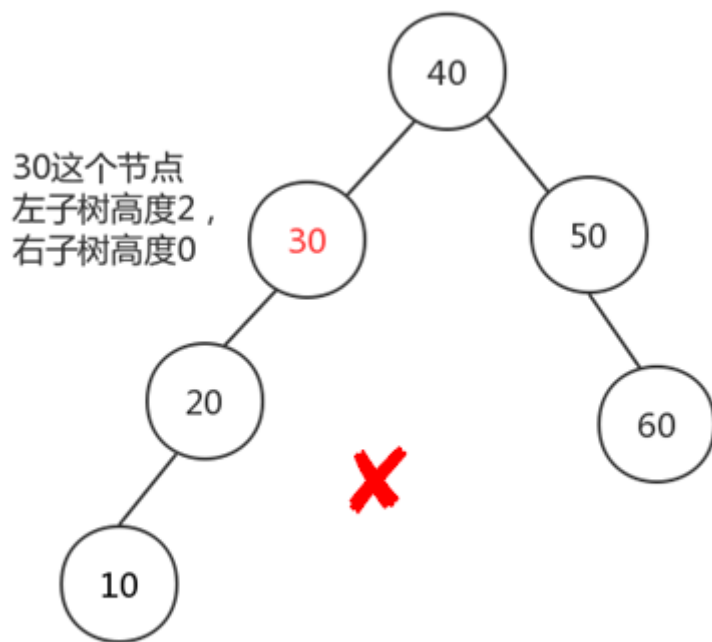
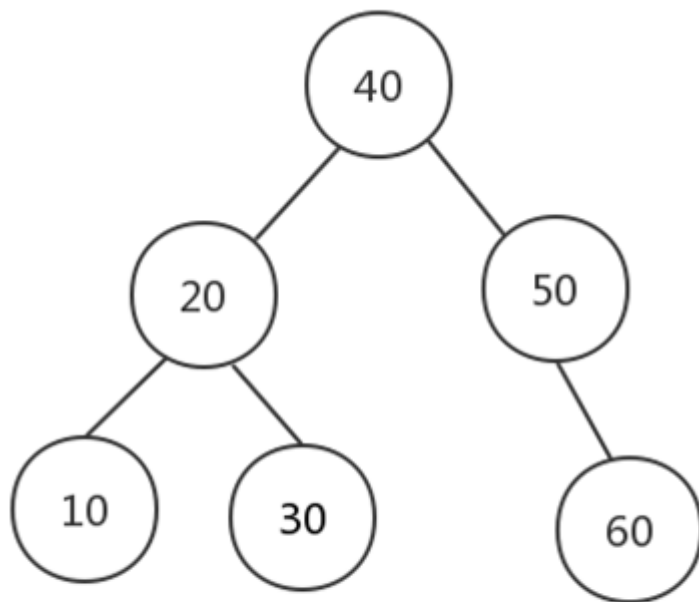


因此若想最大性能地构造一棵二叉查找树，需要这棵二叉查找树是平衡的，从而引出了新的定义——平衡二叉树，或称为AVL树。

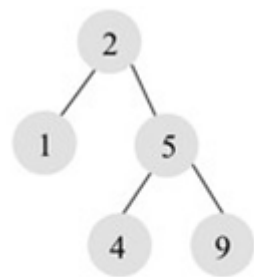
1.1.7.1.6.平衡二叉树 (AVL-树)

它是一棵二叉排序树，它的左右两个子树的高度差（平衡因子）的绝对值不超过1，并且左右两个子树都是一棵平衡二叉树。

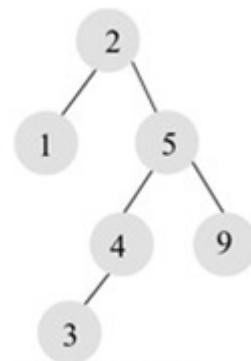
目的：使得树的高度最低，因为树查找的效率决定于树的高度



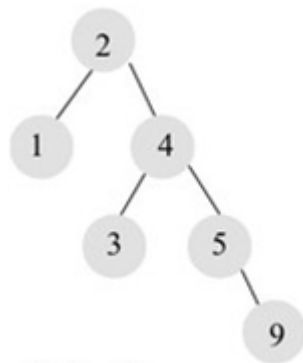
平衡二叉树的查找性能是比较高的，但是维护一棵平衡二叉树的代价是非常大的。通常来说，需要1次或多次左旋和右旋来得到插入、更新和删除后树的平衡性。



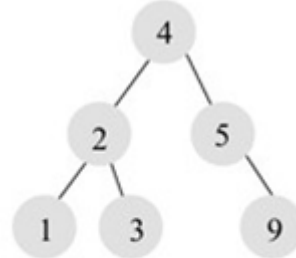
平衡二叉树



插入新键值3



右旋一次



再左旋一次

具体树的旋转见：[算法数据结构体系学习班_马士兵教育官网-IT职业领路人\(mashibing.com\)](http://算法数据结构体系学习班_马士兵教育官网-IT职业领路人(mashibing.com))

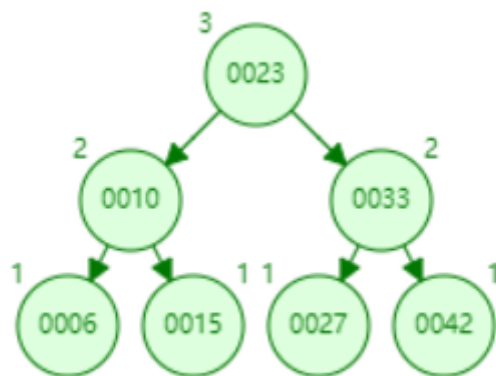
章节11-13

1.1.7.2.B+树

B+ 树是从平衡二叉查找树演化而来（但B+树不是二叉树，而是一个多叉查找平衡树）。

下图就是一颗平衡二叉查找树

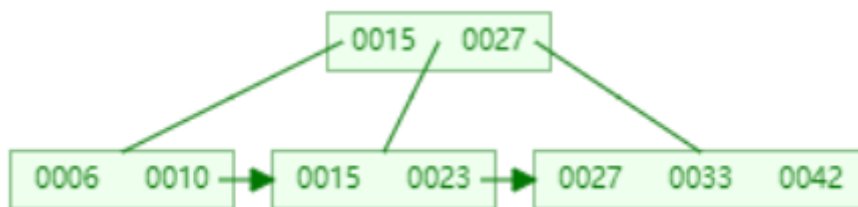
借助网页工具：[Data Structure Visualization \(usfca.edu\)](http://Data Structure Visualization (usfca.edu))



现在我们将其改造成 B+ 树

树的阶数表示一个节点最多能有多少个子节点。

每个叶子页（LeafPage）存储了实际的数据，如下图中有的叶子页就存放了3条数据记录，当然可以更多，叶子节点由小到大（有序）串联在一起，叶子页中的数据也是排好序的；



从AVL到B+树的变化可知，如果节点特别多的话，AVL树的高度远远高于B+树。

我们可以归纳出B+树的几个特征：

- 1、相同节点数量的情况下，B+树高度远低于平衡二叉树；
- 2、非叶子节点只保存索引信息和下一层节点的指针信息，不保存实际数据记录；
- 3、每个叶子页（LeafPage）存储了实际的数据，比如上图中每个叶子页就存放了3条数据记录，当然可以更多，叶子节点由小到大（有序）串联在一起，叶子页中的数据也是排好序的；
- 4、相邻的叶子节点之间用指针相连。

注意：叶子节点中的数据在物理存储上完全可以是无序的，仅仅是在逻辑上有序（通过指针串在一起）。

当然，一棵m阶的B+树完整定义如下：

- 每个节点最多可以有 m 个元素；
- 除了根节点外，每个节点最少有 $(m/2)$ 个元素；
- 如果根节点不是叶节点，那么它最少有 2 个孩子节点；
- 所有的叶子节点都在同一层；
- 非叶子节点只存放关键字和指向下一个孩子节点的索引，记录只存放在叶子节点中；
- 一个有 k 个孩子节点的非叶子节点有 $(k-1)$ 个元素，按升序排列；
- 某个元素的左子树中的元素都比它小，右子树的元素都大于或等于它（二叉排序树的特征）；
- 相邻的叶子节点之间用指针相连。

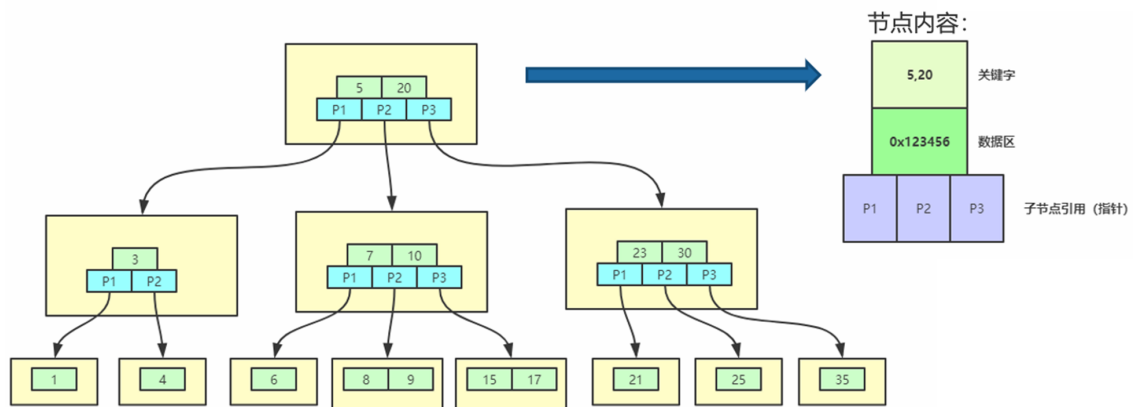
以上知识可以不了解，一般面试不会问到B+树的细节（B+树的插入、B+树的删除、B+树的旋转等等），除非面试岗位就是做数据库实现的。

如果想详细了解，可以找《算法导论》这本书

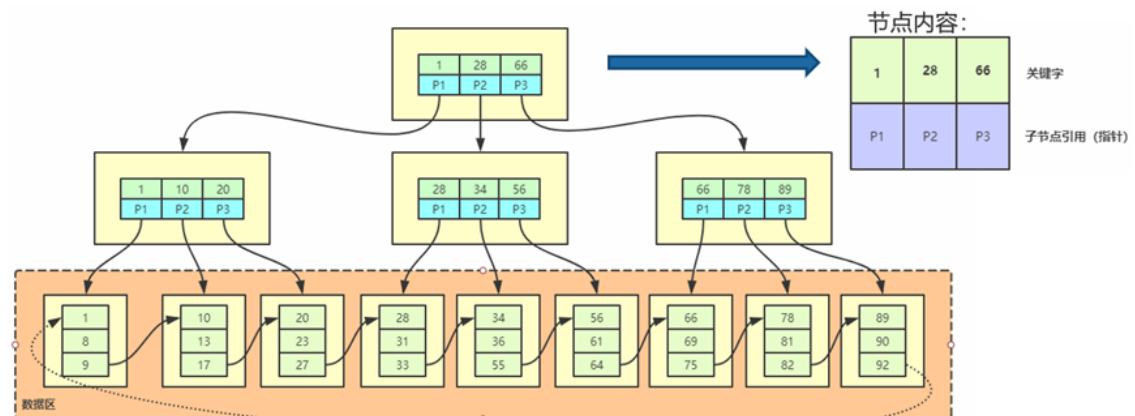
XV	
11.1 直接寻址表	132
11.2 散列表	133
11.3 散列函数	137
11.3.1 除法散列法	138
11.3.2 乘法散列法	138
*11.3.3 全域散列	139
11.4 开放寻址法	142
*11.5 完全散列	146
第12章 二叉查找树	151
12.1 二叉查找树	151
12.2 查询二叉查找树	153
12.3 插入和删除	155
*12.4 随机构造的二叉查找树	158
第13章 红黑树	163
13.1 红黑树的性质	163
13.2 旋转	165
13.3 插入	167
17.3 势能方法	249
17.4 动态表	251
17.4.1 表扩张	251
17.4.2 表扩张和收缩	253
第五部分 高级数据结构	
概述	261
第18章 B树	263
18.1 B树的定义	265
18.2 对B树的基本操作	267
18.3 从B树中删除关键字	272
第19章 二项堆	277
19.1 二项树与二项堆	278
19.1.1 二项树	278
19.1.2 二项堆	279
19.2 对二项堆的操作	281
第20章 斐波那契堆	291

面试问得比较多的可能是B树(也可以是B-树)、B*树，以及为什么选用B+树。

B树与B+树的差别是，B树的非叶子节点也需要存放数据，下图是B树

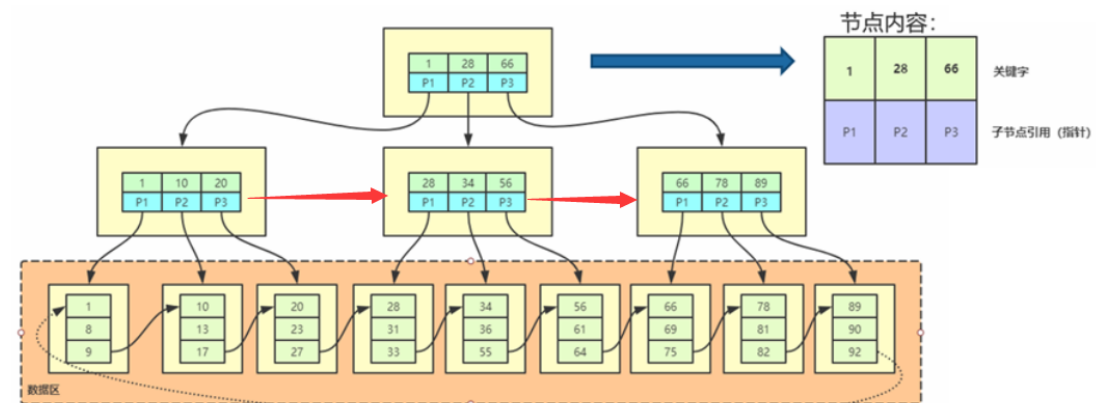


而B+树的话，数据只存在叶子节点上，同时相邻的叶子节点有链表的结构



同时要注意，MySQL中实现的B+树，叶子节点之间的链表是双向链表，这是一个细微的差别。

另外B* *树的话，与B+树的差别就是在非叶子节点之间，也有相互的指针指向。Oracle中使用的是B** 树。



那为什么MySQL不用B树而使用B+树呢？

- 因为B数据每个节点都存储数据，每次查询的数据大小固定，就会造成每次查询返回的数据的条数变少，相同数据规模的情况下B树会增加io次数，而B+树，则数据量较小，一次可以返回多条记录，io次数较少
- 范围查询B+树明显优于B树

1.1.8.MySQL与B+树

为什么关系型数据库都选择了B+树，这个和磁盘的特性有着非常大的关系。

为了提高效率，要尽量减少磁盘I/O。为了达到这个目的，磁盘往往不是严格按需读取，而是每次都会预读，即使只需要一个字节，磁盘也会从这个位置开始，顺序向后读取一定长度的数据放入内存，这个称之为**预读**。

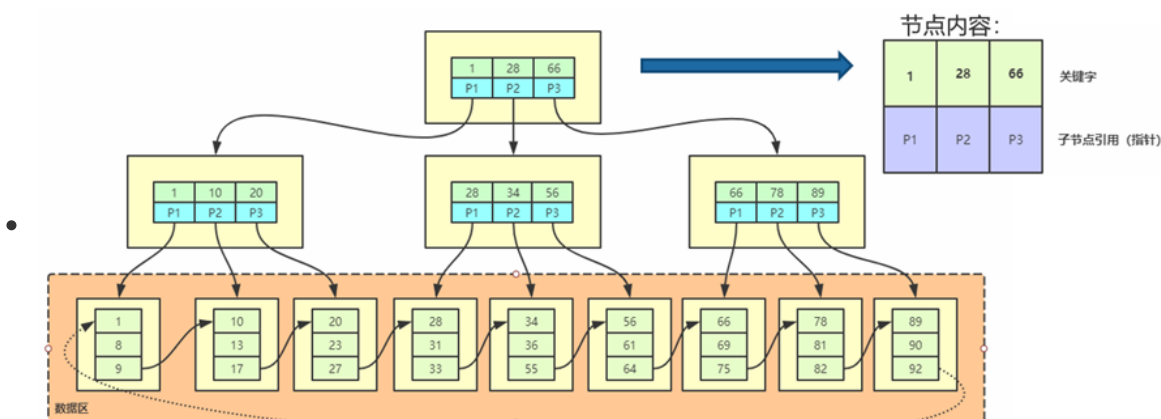
预读的长度一般为页（page）的整倍数。页是计算机管理存储器的逻辑块，硬件及操作系统往往将主存和磁盘存储区分割为连续的大小相等的块，每个存储块称为一页，页大小通常为4k。

按照磁盘的这种性质，如果是一个页存放一个B+树的节点，自然是可以存放很多的数据的，比如InnoDB里，默认定义的B+树的节点大小是16KB，这就是说，假如一个Key是8个字节，那么一个节点可以存放大约1000个Key，意味着B+树可以有1000个分叉。同时InnoDB每一次磁盘I/O，读取的都是16KB的整数倍的数据。也就是说InnoDB在节点的读写上是可以充分利用磁盘顺序IO的高速读写特性。

同时按照B+树逻辑结构来说，在叶子节点一层，所有记录的主键按照从小到大的顺序排列，并且形成了一个双向链表。同一层的非叶子节点也互相串联，形成了一个双向链表。那么在实际读写的时候，很大的概率相邻的节点会放在相邻的页上，又可以充分利用磁盘顺序IO的高速读写特性。

所以我们对MySQL优化的一大方向就是 **尽可能的多让数据顺序读写，少让数据随机读写**。

磁盘顺序读取的效率很高（不需要寻道时间，只需很少的旋转时间），一般来说，磁盘的顺序读的效率是随机读的40到400倍都有可能，顺序写是随机写的10到100倍。



1.1.9.B+树的作用总结

- 在磁盘设备上，通过B+树可以有效的存储数据；
- 所有记录都存储在叶子节点上，非叶子(non-leaf)存储索引(keys)信息；而且记录按照索引列的值由小到大排好了序。
- B+树含有非常高的扇出（fanout），通常超过100，在查找一个记录时，可以有效的减少IO操作；

*扇出：是每个索引节点(Non-LeafPage)指向每个叶子节点(LeafPage)的指针；

*扇出数 = 索引/节点(Non-LeafPage)可存储的最大关键字个数 + 1